



Git – VCS

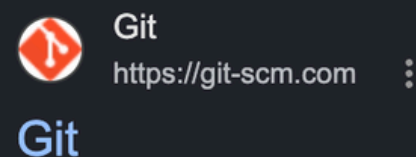
Introduction



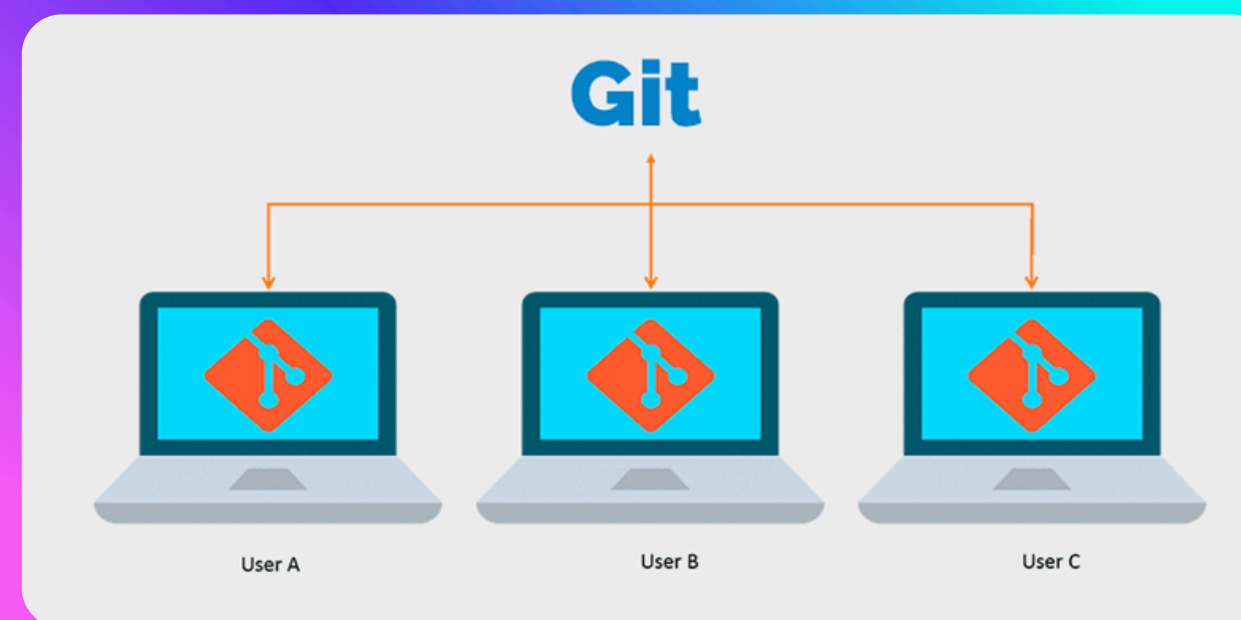


GIT

Git. Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency. Git is lightning fast and has a huge ecosystem of GUIs, hosting services, and command-line tools.



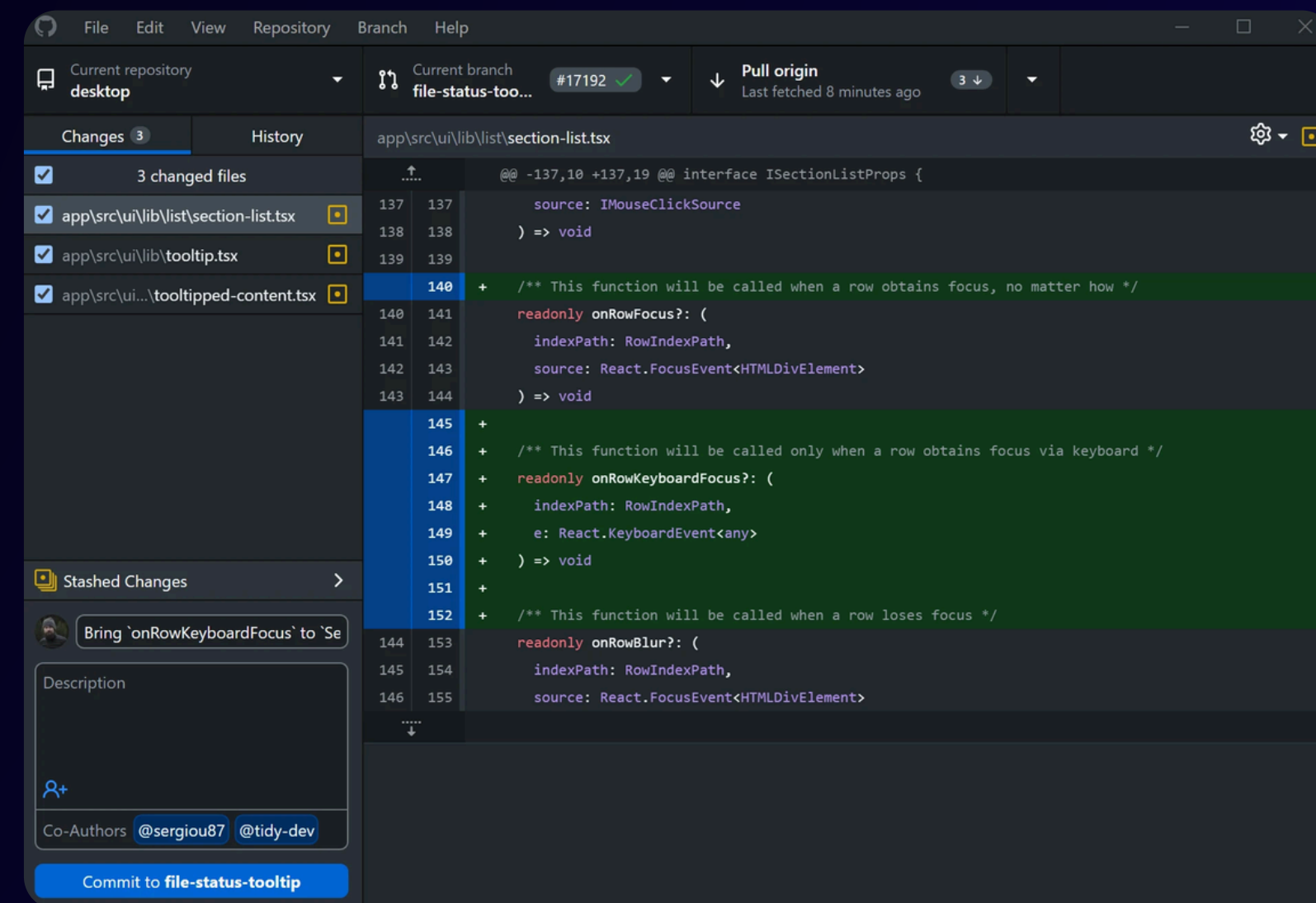
git





Repository

It's a place where you can store your code, your files, and each file's revision history. Repositories can have multiple collaborators and can be either public or private.





Git – pojęcia – **PODSTAWY**

- **VCS (system kontroli wersji)** – narzędzie zapisujące historię zmian w plikach, z możliwością powrotu do dowolnej wersji.
- **Repozytorium (repository)** – folder projektu śledzony przez Git, zawierający pełną historię (folder .git).
- **Rozproszony (distributed) vs scentralizowany (centralized)** – w Gicie każdy klon ma pełną historię (przeciwnie niż w SVN, gdzie historia jest tylko na jednym serwerze).





Git – pojęcia – OBSZARY ROBOCZE

- **Working directory** – pliki, które fizycznie edytujesz na dysku.
- **Staging area / index** – „poczekalnia” na zmiany, które trafią do najbliższego commita (**git add**).
- **Repository (.git)** – zapisana, niezmienna historia commitów.



Git – pojęcia – COMMIT I HISTORIA

- **Commit** – migawka (snapshot) całego stanu projektu w danym momencie, nie tylko różnica (diff).
- **Hash (SHA-1)** – unikalny identyfikator każdego commita.
- **HEAD** – wskaźnik na commit, na którym aktualnie „stoisz” (zwykle czubek bieżącej gałęzi).
- **Parent commit** – commit-rodzic, dzięki czemu historia tworzy graf skierowany.



Git – pojęcia – **BRANCHE**

- **Branch (gałąź)** – lekki wskaźnik na konkretny commit, nie kopia całego projektu.
- **main / master** – domyślna, główna gałąź.
- **Feature branch** – oddzielna gałąź do pracy nad nową funkcją w izolacji.





Differences Between Git, GitHub, and GitLab



Git Version Control System

Function: Tracks changes in source code

Usage: Local development, non-linear workflows

Git Repository Hosting Services

GitHub



Features:

- Git repositories
- Collaborative tools
- GitHub Actions
- Ticket system

Pricing: Free and paid plans

Hosting: Cloud-based

GitLab



Features:

- Git repositories
- CI/CD pipelines
- Self-hosting options

Pricing: Free and paid plans

Hosting: Cloud-based and self-hosted





Dlaczego w ogóle kontrola wersji?

Bez VCS

- projekt_final.docx
- projekt_final2.docx
- projekt_final_NAPRAWDE_OSTATECZNY.docx
- Praca zespołowa przez USB / Dysk Google → nadpisywanie zmian
- Brak odpowiedzi na pytanie: kto i kiedy to zepsuł?

Z Gitem

- + Jeden plik, pełna historia wszystkich wersji
- + Każda zmiana ma autora, czas i opis
- + Równoległa praca wielu osób bez nadpisywania
- + Możliwość cofnięcia do dowolnego momentu



Historia: 2005

Jak Linus Torvalds napisał Gita w 10 dni

Kontekst

Rozwój jądra Linuksa korzystał z komercyjnego systemu BitKeeper. Gdy w 2005 roku firma cofnęła darmową licencję dla społeczności open source, Linus Torvalds — twórca Linuksa — potrzebował alternatywy, i to szybko.

Cel #1

System **rozproszony** — brak jednego centralnego serwera

Cel #2

Szybkość przy tysiącach współtwórców i ogromnej historii

Cel #3

Odporność na uszkodzenia i integralność danych (SHA-1)



Git w ekosystemie VCS

scentralizowane CVS / SVN

Jeden centralny serwer = jedno źródło prawdy. Klienci pobierają tylko aktualny stan, praca offline ograniczona.

rozproszone Git / Mercurial

Każdy klon ma pełną historię projektu. Mercurial powstał niemal równolegle, podobna filozofia.

2008 — GitHub

Platforma hostingowa i społecznościowa zbudowana wokół Gita. To ona uczyniła Gita popularnym poza kręgiem twórców jądra Linuksa. Dziś: GitHub, GitLab, Bitbucket, Azure DevOps.

⚠ Uwaga

Git ≠ GitHub. Git to narzędzie działające lokalnie na Twoim komputerze. GitHub to hosting w chmurze zbudowany wokół Gita — jeden z wielu możliwych.





Workflow

edytujesz
Working Directory
pliki na dysku

`git add`

poczekalnia
Staging Area
index

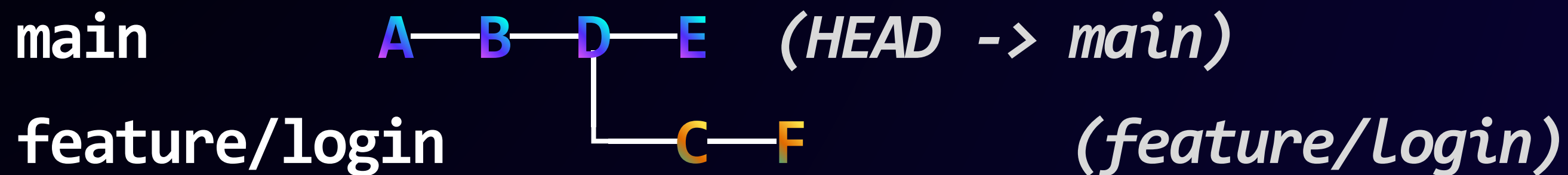
`git commit`

historia
Repozytorium
.git

Staging area pozwala wybrać dokładnie, które zmiany trafią do commita — nie musisz commitować wszystkiego naraz. Można dodać jeden plik, a zostawić drugi na później.



Branch – wskaźnik



*Branch to lekki wskaźnik na konkretny commit. **main** (**dawniej master**) to zwykle gałąź główna, a *feature branch* pozwala pracować nad nową funkcją w izolacji, bez ryzyka dla stabilnego kodu.*



Instalacja Gita

Windows

git-scm.com → instalator **Git for Windows** (zawiera Git Bash). Zwróć uwagę na opcje: domyślny edytor, obsługa końców linii.

Linux (Debian/Ubuntu)

```
sudo apt update  
sudo apt install git
```

macOS

```
brew install git  
# lub:  
xcode-select --install
```

```
$ git --version  
git version 2.45.1
```





Configuration

--system

Dotyczy wszystkich użytkowników komputera. Plik w katalogu instalacji Gita.

--global

Dotyczy bieżącego użytkownika, wszystkich repozytoriów.

`~/.gitconfig`

--local domyślny

Dotyczy tylko jednego repozytorium.

`.git/config` — nadpisuje pozostałe.

priorytet: *local > global > system*



Configuration

```
$ git config --global user.name "Jadwiga Himmel"
```

```
$ git config --global user.email "jaga.himmel@styrta.pl"
```

```
$ git config --global init.defaultBranch main
```

```
$ git config --global core.editor "code --wait"
```

```
$ git config --list
```

Bez **user.name** i **user.email** Git nie pozwoli wykonać commita — te dane trafiają do każdej migawki jako informacja o autorze.



:wq

